МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к лабораторной работе № 2

Тема: «Создание консольного скрипта технологических расчётов: консольный ввод, вычисления по формулам, вывод f-строками»

Вариант А — базовая траектория · Вариант Б — продвинутая траектория
Работа рассчитана на 2 пары (4 академических часа)

Дисциплина: «Программирование и прототипирование ИТ-решений»

Направление подготовки: 09.03.03 Прикладная информатика

Профиль: Бизнес и системная аналитика

Семестр / трудоёмкость: 1 семестр, 4 академических часа (2 пары)

Среда разработки: Visual Studio Code

Форма контроля: отчёт о выполнении работы + устная защита

Обеспеченность ПО: Python 3.11+, Git, VS Code (подготовлены в ЛР № 1)

1. Цель работы

Научиться строить интерактивный консольный скрипт, который запрашивает у пользователя исходные данные, выполняет технологические расчёты по формулам и выводит читаемый, правильно отформатированный результат.

В результате выполнения работы студент научится:

- запрашивать у пользователя числовые значения и преобразовывать ввод в числа (`int()`, `float()`) с обработкой десятичного разделителя «запятая»;
- реализовывать математические вычисления по формулам нефтегазовой отрасли — без «зашитых» готовых ответов;
- форматировать вывод с помощью f-строк: точность, выравнивание, знаки, единицы измерения;
- проверять корректность ввода и устойчиво обрабатывать типичные ошибки пользователя (`try/except`, цикл повторного ввода);
- сверять машинный результат с ручным контрольным пересчётом и документировать это в отчёте.

2. Обязательное ядро (одинаково для обоих вариантов)

- скрипт запускается из командной строки и взаимодействует с пользователем через `input()`;
- числовой ввод преобразуется через `float()` (при необходимости `int()`), а ввод через запятую `3,14` корректно приводится к точке;
- программа вычисляет результат по приведённым формулам, а не печатает заранее заданные числа;

- результат выводится f-строками с указанием единиц измерения и явной точностью (`:.2f`), ключевые поля выровнены;
- в репозитории Git сделано не менее трёх осмысленных коммитов (инициализация → рабочий код → документация);
- в отчёт включён ручной контрольный пересчёт ОДНОГО показателя для ВАШИХ индивидуальных данных;
- выполнены ВСЕ обязательные упражнения на понимание (раздел 9) с фиксацией проверки понимания в отчёте;
- README.md содержит назначение, пример запуска и пример ввода/вывода.

3. Глоссарий: все термины работы

Перед началом работы прочитайте термины. К ним можно возвращаться по ходу работы.

3.1. Консольный ввод и типы данных

Термин	Что это значит
`input(prompt)`	встроенная функция, печатает приглашение и возвращает строку, которую ввёл пользователь. Всегда возвращает `str`.
`float(s)` / `int(s)`	преобразуют строку в вещественное / целое число. При невозможности преобразования бросают `ValueError`.
Десятичный разделитель	символ, отделяющий дробную часть. Python понимает только точку `3.14`; в русской раскладке часто вводят запятую `3,14` — её нужно заменять.
`str.strip()`	убирает пробелы и перевод строки по краям строки; удобно для очистки ввода.
Вещественное число (float)	число с плавающей точкой, например `12.5`. Для технологических величин (дебит, давление) используется именно float.

3.2. Вычисления и f-строки

Термин	Что это значит
f-строка	строка с префиксом `f`, в которой в фигурных скобках можно подставлять значения: `f"Дебит: {q} м³/сут"`.
Спецификатор формата	часть после двоеточия в `{}`: `:.2f` — два знака после запятой; `:8.2f` — выравнивание в поле шириной 8; `:.1%` — проценты.
`round(x, n)`	функция округления числа `x` до `n` знаков после запятой.
Оператор `//`	целочисленное деление: `7 // 2` → `3`.
Оператор `%`	остаток от деления: `7 % 2` → `1`. Используется для персональных вариантов.
Оператор `**`	возведение в степень: `2 ** 3` → `8`.

3.3. Валидация и структура скрипта

Термин	Что это значит
`try / except`	конструкция для перехвата ошибок: если в `try` произошла ошибка (`ValueError`), выполняется блок `except`.
Цикл повторного ввода	`while True:` вокруг ввода: спрашиваем, пока пользователь не

	введёт корректное значение, затем `break`.
`if __name__ == "__main__"`	защита точки входа: код в `main()` выполняется, только когда файл запущен напрямую, а не импортирован.
Константа	значение, не меняющееся в программе (плотности, `g`), записывается ЗАГЛАВНЫМИ буквами в `config.py` или в начале файла.
`main()`	функция — точка входа программы; удобно читать сверху вниз.

4. Необходимое программное обеспечение

Всё окружение подготовлено в лабораторной работе № 1. Убедитесь, что оно на месте:

Инструмент	Назначение	Проверка
Python 3.11+	интерпретатор языка	`python --version`
Visual Studio Code	редактор и терминал	окно VS Code открывается
Git	система контроля версий	`git --version`
Виртуальное окружение	изоляция зависимостей проекта	`(.venv)` в начале строки

5. Общая подготовка (после ЛР № 1)

Шаг 5.1. Проверить инструменты

```
python --version
git --version
```

Зачем это делаем: *Записываем обе версии в отчёт — это подтверждает готовность окружения и помогает воспроизводить результат у другого человека.*

Шаг 5.2. Создать папку проекта

Создайте отдельную папку под этот скрипт (имя — по варианту, см. разделы 6 и 7):

```
mkdir well-calc-basic
```

Зачем это делаем: *Отдельная папка = отдельный проект. В одну папку складываем код, README и настройки; чужие проекты не смешиваем.*

Шаг 5.3. Создать и активировать виртуальное окружение

```
python -m venv .venv
.\.venv\Scripts\Activate.bat
```

Зачем это делаем: *Окружение изолирует библиотеки проекта. После активации в начале строки терминала появляется (.venv).*

Выберите интерпретатор в VS Code: `Ctrl + Shift + P` → `Python: Select Interpreter` → `.venv`.

6. Вариант А (базовый): «Калькулятор технологических показателей скважины»

6.1. Что мы делаем

Создаём консольный скрипт, который запрашивает у пользователя три исходных параметра скважины и по формулам вычисляет: дебит нефти, дебит воды, плотность пластовой жидкости и гидростатическое давление на забое.

Это минимальная, но честная инженерная задача: те же величины считает любой технолог, только в реальных системах данные приходят из базы, а у нас — через `input()`.

6.2. Индивидуальные исходные данные

Чтобы работа была индивидуальной, исходные параметры вычисляются из вашего номера `N` (номер по журналу группы; если неизвестен — последние две цифры номера зачётки). Формулы:

```
N  = ваш номер по журналу (или 2 последние цифры зачётки)

Qж = 45 + 2·N         (дебит жидкости, м³/сут)
W  = 0.10 + 0.015·(N % 20)   (обводнённость, доли ед.)
H  = 1800 + 40·(N % 15)      (глубина, м)
```

`N % 20` и `N % 15` — остаток от деления. Подставьте свой `N` и вычислите `Qж`, `W`, `H` на бумаге — это ваши данные для запуска и ручного контрольного пересчёта. Запишите их в отчёт.

Шаг 1. Создать проект

```
mkdir well-calc-basic
cd well-calc-basic
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Создайте в корне папки файл `main.py` (можно без сложной структуры пакетов — вариант А одностраничный).

Шаг 2. Написать код

Файл `main.py` — используйте этот каркас как основу:

```
"""Калькулятор технологических показателей нефтяной скважины (вариант А)."""

# ---- константы ----
WATER_DENSITY = 1000.0   # плотность воды, кг/м³
OIL_DENSITY = 850.0      # плотность нефти, кг/м³
G = 9.81                 # ускорение свободного падения, м/с²


def input_float(prompt: str) -> float:
    """Запрашивает число, принимает и точку, и запятую."""
    while True:
        raw = input(prompt).strip().replace(",", ".")
        if raw == "":
            print("Пустая строка — введите число.")
            continue
```

```
        try:
            return float(raw)
        except ValueError:
            print(f"'{raw}' не похоже на число. Повторите ввод.")


def main() -> None:
    print("РАСЧЁТ ТЕХНОЛОГИЧЕСКИХ ПОКАЗАТЕЛЕЙ СКВАЖИНЫ")
    print("=" * 44)

    q_liq = input_float("Дебит жидкости Qж, м³/сут: ")
    w_cut = input_float("Обводнённость W, доли ед. (0..1): ")
    depth = input_float("Глубина скважины H, м: ")

    # 1) дебиты нефти и воды
    q_oil = q_liq * (1 - w_cut)
    q_water = q_liq * w_cut
    # 2) плотность пластовой жидкости (смеси)
    rho_mix = OIL_DENSITY * (1 - w_cut) + WATER_DENSITY * w_cut
    # 3) гидростатическое давление столба жидкости
    p_hydro = rho_mix * G * depth / 1_000_000   # Па -> МПа

    print()
    print("РЕЗУЛЬТАТЫ")
    print("-" * 44)
    print(f"Дебит нефти:       {q_oil:8.2f} м³/сут")
    print(f"Дебит воды:        {q_water:8.2f} м³/сут")
    print(f"Плотность смеси:   {rho_mix:8.1f} кг/м³")
    print(f"Забойное давление: {p_hydro:8.2f} МПа")


if __name__ == "__main__":
    main()
```

Зачем это делаем: *`replace(",", ".")` спасает, когда пользователь вводит `0,25` вместо `0.25`. Формулы записаны в явном виде — по коду видно, что результат действительно вычисляется, а не печатается «из головы».*

Шаг 3. Запустить и проверить

```
python main.py
```

Введите свои значения Qж, W, H (из п. 6.2). Пример вывода (для `N = 5`: Qж = 55, W = 0.175, H = 2000):

```
РАСЧЁТ ТЕХНОЛОГИЧЕСКИХ ПОКАЗАТЕЛЕЙ СКВАЖИНЫ
============================================
Дебит жидкости Qж, м³/сут: 55
Обводнённость W, доли ед. (0..1): 0,175
Глубина скважины H, м: 2000

РЕЗУЛЬТАТЫ
--------------------------------------------
Дебит нефти:          45.38 м³/сут
Дебит воды:            9.63 м³/сут
```

```
Плотность смеси:     876.2 кг/м³
Забойное давление:    17.19 МПа
```

Зачем это делаем: *Обратите внимание: запятая в `0,175` была принята, потому что мы её заменили на точку. Правильность цифр проверим вручную на следующем шаге.*

Шаг 4. Ручной контрольный пересчёт (обязательно)

Пересчитайте на калькуляторе вручную ОДИН показатель — например, дебит нефти — с подстановкой своих чисел и приведите в отчёте полную цепочку:

```
Qн = Qж · (1 − W) = 55 · (1 − 0,175) = 55 · 0,825 = 45,375 м³/сут
```

Сверьте: ручной результат должен совпасть со строкой «Дебит нефти» программы (с учётом округления до двух знаков — `45.38`).

Зачем это делаем: *Ручной пересчёт — гарантия того, что вы поняли формулу, а не просто «запустили код». Именно его спрашивают на защите.*

Шаг 5. Закоммитить в Git

```
git init
git add main.py
git commit -m "Вариант А: ввод, расчёт, вывод"
```

Добавьте `README.md` (назначение + пример запуска) и сделайте отдельный коммит:

```
git add README.md
git commit -m "Добавить README с примером запуска"
```

Зачем это делаем: *Два коммита с осмысленными сообщениями показывают историю разработки: сначала рабочий код, затем документация.*

7. Вариант Б (продвинутый): «Инженерный калькулятор технологического режима»

7.1. Что мы делаем

Расширяем вариант А до инструмента технолога: добавляем пластовое и забойное давления, вычисляем депрессию и коэффициент продуктивности, организуем жёсткую валидацию ввода и выводим результат в виде аккуратной таблицы. Логика разнесена по файлам, как в настоящем проекте.

7.2. Индивидуальные исходные данные

К данным варианта А добавляются два давления (тот же номер `N`):

```
Рпл  = 20.0 + 0.3·(N % 10)   (пластовое давление, МПа)
Рзаб = 12.0 + 0.2·(N % 12)   (забойное давление, МПа)
```

Выпишите все пять чисел (`Qж`, `W`, `H`, `Рпл`, `Рзаб`) — это ваш индивидуальный набор.

Шаг 1. Создать проект

```
mkdir well-calc-advanced
cd well-calc-advanced
```

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Создайте структуру пакета:

```
well-calc-advanced/
├── .gitignore
├── README.md
└── src/
    └── well_calc/
        ├── __init__.py
        ├── config.py
        └── main.py
```

Шаг 2. Написать код

Файл `src/well_calc/__init__.py`:

```
"""Инженерные расчёты технологического режима скважины."""

__version__ = "0.2.0"
```

Файл `src/well_calc/config.py` — константы отдельно от логики:

```
"""Конфигурация и физические константы проекта."""
WATER_DENSITY = 1000.0    # плотность воды, кг/м³
OIL_DENSITY = 850.0       # плотность нефти, кг/м³
G = 9.81                  # ускорение свободного падения, м/с²
PA_TO_MPA = 1_000_000     # Па в одном МПа
```

Файл `src/well_calc/main.py`:

```
"""Инженерный калькулятор технологического режима (вариант Б)."""
from well_calc import config


def input_number(prompt: str, lo: float, hi: float) -> float:
    """Числовой ввод в диапазоне [lo; hi] с защитой от ошибок."""
    while True:
        raw = input(prompt).strip().replace(",", ".")
        try:
            value = float(raw)
        except ValueError:
            print(f"'{raw}' не число. Повторите.")
            continue
        if lo <= value <= hi:
            return value
        print(f"Значение должно быть в диапазоне [{lo}; {hi}].")


def calc_q_oil(q_liq: float, w_cut: float) -> float:
    return q_liq * (1 - w_cut)


def calc_q_water(q_liq: float, w_cut: float) -> float:
    return q_liq * w_cut
```

```
def calc_rho_mix(w_cut: float) -> float:
    return config.OIL_DENSITY * (1 - w_cut) + config.WATER_DENSITY * w_cut


def calc_p_hydro(rho_mix: float, depth: float) -> float:
    return rho_mix * config.G * depth / config.PA_TO_MPA


def calc_drawdown(p_res: float, p_bhp: float) -> float:
    return p_res - p_bhp   # депрессия на пласт


def main() -> None:
    print("ИНЖЕНЕРНЫЙ КАЛЬКУЛЯТОР ТЕХНОЛОГИЧЕСКОГО РЕЖИМА")
    print("=" * 52)

    q_liq = input_number("Дебит жидкости Qж, м³/сут: ", 0.0, 1000.0)
    w_pct = input_number("Обводнённость, %: ", 0.0, 100.0)
    depth = input_number("Глубина скважины H, м: ", 0.0, 5000.0)
    p_res = input_number("Пластовое давление Рпл, МПа: ", 0.0, 100.0)
    p_bhp = input_number("Забойное давление Рзаб, МПа: ", 0.0, 100.0)

    w_cut = w_pct / 100.0                # % -> доли ед.
    q_oil = calc_q_oil(q_liq, w_cut)
    q_water = calc_q_water(q_liq, w_cut)
    rho_mix = calc_rho_mix(w_cut)
    p_hydro = calc_p_hydro(rho_mix, depth)
    drawdown = calc_drawdown(p_res, p_bhp)

    print()
    print("СВОДКА РАСЧЁТА")
    print("-" * 52)
    print(f"{'Показатель':<28}{'Значение':>14}")
    print("-" * 52)
    print(f"{'Дебит нефти':<28}{q_oil:>12.2f} м³/сут")
    print(f"{'Дебит воды':<28}{q_water:>12.2f} м³/сут")
    print(f"{'Плотность смеси':<28}{rho_mix:>12.1f} кг/м³")
    print(f"{'Забойное (гидрост.) давление':<28}{p_hydro:>12.2f} МПа")
    print(f"{'Депрессия на пласт':<28}{drawdown:>12.2f} МПа")

    if drawdown < 0:
        print("(!) Забойное давление выше пластового — вероятно фонтанирование.")
    else:
        productivity = q_liq / drawdown
        print(f"{'Коэфф. продуктивности J':<28}{productivity:>12.3f}
м³/(сут·МПа)")


if __name__ == "__main__":
    main()
```

Зачем это делаем: *Каждая формула — отдельная функция: проще читать, тестировать и переиспользовать. Константы вынесены в config.py — менять*

плотности можно, не трогая логику. `input_number` ограничивает диапазон и не даёт ввести отрицательную глубину.

Шаг 3. Запустить и проверить

```
$env:PYTHONPATH="src"
python -m well_calc.main
```

Пример диалога (для `N = 5`: Рпл = 22.4, Рзаб = 13.6):

```
ИНЖЕНЕРНЫЙ КАЛЬКУЛЯТОР ТЕХНОЛОГИЧЕСКОГО РЕЖИМА
==================================================
Дебит жидкости Qж, м³/сут: 55
Обводнённость, %: 17,5
Глубина скважины H, м: 2000
Пластовое давление Рпл, МПа: 22,4
Забойное давление Рзаб, МПа: 13,6

СВОДКА РАСЧЁТА
--------------------------------------------------
Показатель                          Значение
--------------------------------------------------
Дебит нефти                            45.38 м³/сут
Дебит воды                              9.63 м³/сут
Плотность смеси                        876.2 кг/м³
Забойное (гидрост.) давление           17.19 МПа
Депрессия на пласт                      8.80 МПа
Коэфф. продуктивности J                6.250 м³/(сут·МПа)
```

Шаг 4. Ручной контрольный пересчёт

В варианте Б покажите ручной расчёт депрессии и коэффициента продуктивности:

```
ΔP = Рпл − Рзаб = 22,4 − 13,6 = 8,80 МПа
J = Qж / ΔP = 55 / 8,80 = 6,25 м³/(сут·МПа)
```

Шаг 5. Закоммитить в ветке Git

```
git init
git switch -c feature/calculator
git add .
git commit -m "Добавить конфигурацию и функции расчёта"
git add src/well_calc/main.py
git commit -m "Добавить ввод, валидацию и таблицу результатов"
git switch -c main
git merge feature/calculator
```

8. Обязательные упражнения

Семь упражнений выполняют ВСЕ студенты — и в варианте А, и в варианте Б. Скриншоты всех упражнений сразу добавляйте в отчет.

Упражнение 1. Ввод в процентах

Переделайте ввод обводнённости так, чтобы пользователь вводил ПРОЦЕНТЫ (`17.5`), а программа сама переводила их в доли делением на 100.

Зачем это делаем: *Инженер мыслит процентами. Главное, чтобы физическая обводнённость при этом не изменилась.*

Покажите двумя запусками, что ввод `17.5` (проценты) и `0.175` (доли) дают одинаковый дебит нефти.

Упражнение 2. Прогноз до запуска

Перед запуском выпишите на бумаге ВСЕ числа, которые программа должна вывести для ваших индивидуальных данных, с подстановкой значений в каждую формулу. Затем запустите и сверьте.

Зачем это делаем: *Прогноз до запуска — самый сильный признак понимания: его невозможно «скачать» или сгенерировать.*

Проверка понимания (в отчёт): В отчёте приведите таблицу «прогноз | факт» по каждому показателю. Расхождение должно быть только в округлении.

Упражнение 3. Провокационные вводы

Последовательно введите: пустую строку, текст `abc`, отрицательное число, дробь через запятую `0,25` и очень большое число `99999`. Для каждого ввода снимите скриншот реакции программы.

Зачем это делаем: *Намеренно сломанный сценарий доказывает, что валидация реально работает, а не «написана для галочки».*

Проверка понимания (в отчёт): К каждому из пяти скриншотов укажите СТРОКУ вашего кода, которая обработала этот ввод.

Упражнение 4. Граничные значения

Запустите программу с обводнённостью `W = 0` (чистая нефть) и `W = 1` (чистая вода). Объясните полученные цифры физически.

Зачем это делаем: *Крайние случаи ловят ошибки в формулах: при `W = 1` дебит нефти обязан быть равен нулю.*

Проверка понимания (в отчёт): В отчёте запишите результаты обоих запусков и физический смысл каждой строки.

Упражнение 5. Текстовая диаграмма структуры жидкости

Добавьте в вывод две текстовые полосы из символов `#`, длина которых пропорциональна доле нефти и воды:

```
Нефть:  ##########  82.5 %
Вода:   ##          17.5 %
```

Зачем это делаем: *Диаграмма делает результат наглядным; длина полосы — это доля, умноженная на ширину шкалы.*

Проверка понимания (в отчёт): Объясните в отчёте, по какой формуле вычисляется число символов `#` в одной полосе, и почему оно может не совпадать с процентами «лоб в лоб».

Упражнение 6. Новая формула — перевод в тонны

Добавьте расчёт суточного объёма нефти в тоннах:

```
m_oil = q_oil * config.OIL_DENSITY / 1000   # т/сут
```

Зачем это делаем: *Массовый дебит нужен для планов перекачки: формула связывает объём, плотность и массу.*

Проверка понимания (в отчёт): Пересчитайте эту формулу вручную для своих данных и приведите цепочку вычислений.

Упражнение 7. Вырожденный случай (без падения программы)

Обработайте крайний случай, когда вычисление теряет смысл:

- вариант А — при `W = 1` дебит нефти равен нулю: выведите честное значение и пояснение «нефти нет»;
- вариант Б — при `ΔP = 0` деление `q_liq / drawdown` даёт `ZeroDivisionError`: добавьте проверку и выведите «депрессия нулевая, J не определён».

Зачем это делаем: *Устойчивость к вырожденным данным — признак зрелого кода, а не «пример из тетради».*

Проверка понимания (в отчёт): Приложите скриншот, на котором программа корректно (без traceback) обрабатывает этот случай.

10. Пример оформления отчёта

- Титульная часть: дисциплина, тема, ФИО, группа, дата, вариант, номер по журналу `N`;
- Цель работы;
- Оборудование и ПО с версиями (Python, Git, VS Code);
- Индивидуальные исходные данные: ваш `N` и вычисленные `Qж, W, H, Рпл, Рзаб`;
- Ход работы: команды и скриншоты создания проекта, запуска;
- Результаты: вывод программы;
- Ручной контрольный пересчёт одного показателя;
- Ответы на контрольные вопросы (раздел 12);
- Вывод.

Пример начала отчёта (вариант А):

```
----------------------------------------
Отчёт по лабораторной работе № 2
Дисциплина: Программирование и прототипирование ИТ-решений
Вариант А · Номер по журналу N = 5

Исходные данные:
  Qж = 55 м³/сут, W = 0,175, H = 2000 м

Контрольный пересчёт:
  Qн = Qж · (1 − W) = 55 · 0,825 = 45,375 ≈ 45,38 м³/сут
  (совпадает с выводом программы)
----------------------------------------
```

11. Частые ошибки и их решение

Симптом	Причина и решение
ValueError: could not convert string to float	ввод не число или содержит запятую: добавьте `.replace(",", ".")` и оберните в `try/except`.
ZeroDivisionError при делении	депрессия `ΔP` равна нулю: добавьте проверку `drawdown == 0` перед делением.
Программа печатает `45.375` вместо `45.38`	не применён спецификатор формата: используйте `:.2f` в f-строке или `round(x, 2)`.
Скрипт ничего не делает при запуске через `python main.py`	нет вызова `main()` или он спрятан под `if __name__ == "__main__"` и файл импортируется, а не запускается.
Отступы «съехали» после копирования кода	код скопирован с табуляциями: выровняйте отступы пробелами (4 пробела = 1 уровень).
Коммит не создался, git ругается «nothing to commit»	файлы не добавлены в индекс: сначала `git add .`, затем `git commit`.

12. Контрольные вопросы

Ответьте на каждый вопрос письменно в отчёте, своими словами, с примерами из вашего кода. Ответы «да/нет» или копирование формулировок из методички не засчитываются — нужно показать понимание.

Вопрос 1. Какого типа значение возвращает функция `input()`? Как в вашем скрипте это значение превращается в число? Что произойдёт, если пользователь введёт `abc`?

Вопрос 2. Почему при вводе десятичной дроби через запятую (`0,25`) программа без `replace(",", ".")` падает с ошибкой?

Вопрос 3. Зачем в функции `input_float` используется цикл `while True:` с `break`? Почему нельзя заранее записать условие цикла, как в обычном `while`?

Вопрос 4. Что делает конструкция `try/except`?